

In [5]:

```
#OOPS Concepts
class Product:
    def __init__(self, id, name, price):
        self.id = id
        self.name = name
        self.price = price
    def display_product(self):
        print(self.id, self.name, self.price)

p1 = Product(101, "PC", 45000)
p2 = Product(102, "Mouse", 450)

p1.display_product()
p2.display_product()
```

```
101 PC 45000
102 Mouse 450
```

In [9]:

```
class Product:
    count = 0
    def __init__(self, id, name, price):
        self.id = id
        self.name = name
        self.price = price
        Product.count = Product.count + 1
    def display_product(self):
        print(self.id, self.name, self.price)

p1 = Product(101, "PC", 45000)
p2 = Product(102, "Mouse", 450)
p3 = Product(103, "Laptop", 33450)

p1.display_product()
p2.display_product()
p3.display_product()

print("Number of object of product class = ", Product.count)
```

```
101 PC 45000
102 Mouse 450
103 Laptop 33450
Number of object of product class = 3
```

In [16]:

```
class Employee:
    def __init__(self):
        print('Non-paramterized Constructor')

    def __init__(self, name , dept):
        print('Paramterized Constructor')
        self.name = name
        self.dept = dept

    def display(self, name, dept):
        print('Welcome ', name , 'to ', dept , ' department')

emp1 = Employee('Rohan', 'Accounts')
emp1.display(emp1.name, emp1.dept)

emp2 = Employee('Joe', 'HR')
emp1.display(emp2.name, emp2.dept)
```

Paramterized Constructor
Welcome Rohan to Accounts department
Paramterized Constructor
Welcome Joe to HR department

In [17]:

```
class Student:
    roll_no = '101'
    name = 'Joe'

    def display(self):
        print(self.roll_no, self.name)

s1 = Student()
s1.display()
```

101 Joe

In [18]:

```
class Student:
    def __init__(self):
        print("Default constructor 1")
    def __init__(self):
        print("Default constructor 2")
s1 = Student()
```

Default constructor 2

In [27]:

```
print(emp1.name, emp1.dept)
print(emp2.name, emp2.dept)
print(getattr(emp1, 'name'))
print(getattr(emp1, 'dept'))

setattr(emp1, 'name', 'Tom')
print(emp1.name, emp1.dept)

setattr(emp2, 'dept', 'UIC')
print(emp2.name, emp2.dept)
```

Tom Accounts
Joe HR
Tom
Accounts
Tom Accounts
Joe UIC

In [31]:

```
if hasattr(emp1, 'name'):
    print('Employee has attribute name')
else:
    print('Employee has no attribute name')
```

Employee has attribute name

In [32]:

```
delattr(emp1, 'name')
```

In [33]:

```
print(hasattr(emp1, 'name'))
```

False

In [34]:

```
print(hasattr(emp2, 'name'))
```

True

In [35]:

```
print(emp1.__dict__)
```

{'dept': 'Accounts'}

In [41]:

```
print(emp2.__dict__)
print(emp2.__module__)
print(emp2.__doc__)
```

{'name': 'Joe', 'dept': 'UIC'}
__main__
None

In [43]:

```

class Person:
    def aadhar(self):
        print("Person is having a valid aadhar")

class Employee (Person):
    def info(self):
        print('Employee of our department')

e1 = Employee()
e1.info()
e1.aadhar()

```

Employee of our department
 Person is having a valid aadhar

In [46]:

```

class Person:
    def aadhar(self):
        print("Person is having a valid aadhar")

class Employee (Person):
    def info(self):
        print('Employee of our department')

class Relative (Employee):
    def details(self):
        print("My Relative is an Employee")

e1 = Employee()
e1.info()
e1.aadhar()

print("=====")
r1 = Relative()
r1.info()
r1.aadhar()
r1.details()

```

Employee of our department
 Person is having a valid aadhar
 =====
 Employee of our department
 Person is having a valid aadhar
 My Relative is an Employee

In [47]:

```
class Calculation1:
    def Summation(self,a,b):
        return a+b;

class Calculation2:
    def Multiplication(self,a,b):
        return a*b;

class Derived(Calculation1,Calculation2):
    def Divide(self,a,b):
        return a/b;

d = Derived()
print(d.Summation(10,20))
print(d.Multiplication(10,20))
print(d.Divide(10,20))
```

30
200
0.5

In [51]:

```
print(issubclass(Derived, Calculation1))
print(issubclass(Derived, Calculation2))
print(issubclass(Calculation2, Calculation1))
```

True
True
False

In [55]:

```
print(isinstance(d,Derived))
print(isinstance(d,Calculation1))
print(isinstance(d,Calculation2))

print(isinstance(d,Employee))
```

True
True
True
False

In [57]:

```
#method overriding
class Calculation3:
    def operation(self,a,b):
        return a+b;

class Calculation4 (Calculation3):
    def operation(self,a,b):
        return a-b;

obj = Calculation4()
print(obj.operation(9,3))
```

6

In [61]:

```
class Bank:
    def getroi(self):
        return 4.0

class SBI (Bank):
    def getroi(self):
        return 4.5

class ICICI (Bank):
    def getroi(self):
        return 6.0

class HDFC (Bank):
    def __init__(self):
        print("HDFC")

s1 = SBI()
i1 = ICICI()
h1 = HDFC()

print(s1.getroi())
print(i1.getroi())
print(h1.getroi())
```

HDFC

4.5

6.0

4.0

In []: